Evidence-Gated Prompt Optimization on a Synthetic Attractor Scaffold: A Preprint of the `prompt-optimization` Toolkit

Author Name[1] Collaborator Name[2]
[1]Affiliation, [2]Affiliation
placeholder@example.org

August 28, 2026

Abstract

How much of "this prompt is better" can a prompt-optimization pipeline actually claim? *prompt-optimization* is a reproducible Python scaffold for studying prompt rate-distortion, behavioral attribution, and candidate selection. Its pipeline scores discrete prompt candidates against a set of named behavioral *attractors* in a synthetic embedding model, attributes score differences to prompt components, and selects candidates via an empirical distortion-rate frontier, Pareto non-dominance, and *mandatory hard gates* that no score may override. This preprint documents that mechanism and architecture, gives one real worked example (scoring the candidate files of an independent skill repository), and – the load-bearing part – reports an explicit *Evaluation Status*: the toolkit validates *implementation invariants only*. Its numbers are synthetic attractor outputs, not measurements of any LLM. No real held-out model evaluation exists yet; no pre-registered protocol exists yet; and the pipeline currently has no architectural seam at which a real evaluator could *dispose* of synthetic proposals. We state these gaps as findings, in the spirit that characterizes this line of research: behavioral performance must dominate embedding similarity, and an absence of evidence is itself a reportable result.

Automatic prompt optimization has matured from token-level search [Shin et al., 2020] to an active subfield of generate-and-search [Zhou et al., 2022, Yang et al., 2023, Fernando et al., 2023, Guo et al., 2023, Pryzant et al., 2023, Khattab et al., 2023, Yuksekgonul et al., 2024], with scoring functions that range from task accuracy to optimizer-LLM judgment. A naive version of this loop has a quiet failure mode: the ranking step slowly becomes "how geometrically close is this candidate to a hand-defined target embedding," and the whole enterprise quietly turns into a search for cosine neighbors of one vector. The present project was motivated by the observation that *embedding similarity is not behavioral equivalence* [Kornblith et al., 2019]: a candidate can sit near a target direction and be behaviorally inert, or sit far away and be behaviorally decisive. The design stance of `prompt-optimization` is therefore that *behavioral performance must dominate embedding similarity*, and that any ordering signal from embeddings must be re-grounded on real held-out evaluation before it becomes a claim.

This preprint documents the toolkit as a *research scaffold* for developing and checking that ordering logic cheaply and reproducibly, before any expensive real-model experiment is run. It makes three contributions, in the honest-scope sense of `specimen`-style evaluation:

1. **A vocabulary and selection discipline** – named behavioral attractors, basin probabilities, reliability and a deliberately-admitted-toy distortion proxy, plus empirical distortion-rate and Pareto selection with mandatory hard gates that no scalar score may override.

2. **A shipped architecture and one real worked example** – three subpackages (`metrics`, `attribution`, `context`), a stdlib-only CLI (`prompt-opt scan` / `plan`), and a genuine transcript of the tool being pointed at an independent skill repository.

3. **An honest evaluation status** – a classification of exactly what the shipped evidence licenses and what it does not, which doubles as a concrete fix list for turning this scaffold into a measurement device.

The evaluation-status section is the point of this paper. The rest of the paper is machinery; Section is where the reader should look if they want to know what this toolkit can and cannot yet claim. Related work is organized by mechanism, not chronology. For each cluster we state what the cited work *establishes* and what it does *not*, and we flag the one correction to the project's own notes that surfaced during the literature scan for this preprint.

Discrete prompt search (generate-and-search). Zhou et al. [2022] introduced LLM-generated candidate prompts scored on task accuracy *(automatic prompt engineering)*; Yang et al. [2023] framed the optimizer itself as an LLM that proposes candidates from a scored history, and reported the now-celebrated case where a short phrase ("take a deep breath") beat longer instructions. Later scale-limits work [Zhang et al., 2024] shows the optimizer's size and human-written meta-instructions strongly condition these results — a direct warning against letting a scaffold rank without stating its evaluator. Shin et al. [2020] searched trigger tokens via gradients; Fernando et al. [2023] evolved prompts; Guo et al. [2023] reused evolutionary operators; Pryzant et al. [2023] treated natural-language critiques as "gradients" with beam search; Khattab et al. [2023] compiled declarative LM programs; Yuksekgonul et al. [2024] propagated textual gradients through computation graphs; and Bystroński et al. [2025] searched a continuous latent space and decoded candidates. What this cluster establishes: discrete or latent prompt candidates can be searched and scored programmatically. What it does *not*: it does not establish *which wording does the most behavioral work per token*, and much of it scores on the same model that proposes, leaving an evaluator-leakage risk that Zhang et al. [2024] makes concrete.

Compression and information-theoretic bounds. Jiang et al. [2023] estimate token importance to compress existing prompts for inference cost; the strongest precedent for this project's distortion-rate posture is Nagle et al. [2024] ("Fundamental Limits of Prompt Compression"), which derives the distortion-rate function for black-box prompt compression as a linear program. That paper establishes a *theoretical* bound; this toolkit's `rate_distortion_frontier` is an *empirical* lower envelope, a strictly weaker object. Tishby et al. [2000] provides the Information Bottleneck complement (compress input while preserving relevant information), and Xie et al. [2021] supplies the theoretical intuition for why specific wording narrows the effective output distribution. None of these operations turns a synthetic simulator into evidence about a real model.

Model selection and uncertainty. Akinwande et al. [2023] show that a language-model prior plus a discrete prompt yields PAC-Bayes generalization bounds usable for model selection – relevant because selection-confidence needs a penalty for optimizing on the selection set only. Sabbatella et al. [2023] formulate discrete prompt tuning as combinatorial Bayesian optimization over a continuous surrogate; Zhou and Ou [2026] treat prompts as posterior samples with a language-model prior (a 2026 result this project cites as *research option*, not evidence). Angelopoulos and Bates [2021] give the split-conformal toolkit that the project's `conformal_radius` mirrors in code. These works

establish techniques for honest uncertainty; this project ships the *machinery*, not the calibrated real-world numbers.

Mechanism: why short words matter. Kornblith et al. [2019] show that representation-similarity measures such as CKA do not track task-relevant behavioral similarity — the paper this project's "similarity $\neq$ behavior" stance leans on. Steering work [Turner et al., 2023, Li et al., 2023a, Zou et al., 2023, Panickssery et al., 2023, Meng et al., 2022] operates by causal intervention on activations, *not* prompt wording; this project borrows only the measurement instinct (project a candidate's effect onto a declared direction) and does not claim access to any production model's activations. Bartoszcze et al. [2025] provide the recent survey.

Evaluation robustness. Wallace et al. [2024] show that the priority among system/developer/user text is itself learned behavior; Li et al. [2023b] show instruction-following degrades under conflicting embedded text (so "off-target" should include unauthorized or unsafe behavior, not only semantic competitors); Wu et al. [2023] show that apparent reasoning can rely on default-condition shortcuts, motivating held-out and counterfactual evaluation rather than more examples from the same template.

Novelty claim (scoped and checkable). To our knowledge, no other shipped, reproducible scaffold combines (a) scoring discrete prompt candidates against *named behavioral attractors*, (b) empirical distortion-rate *plus* Pareto selection, (c) *mandatory* hard gates that veto candidates regardless of score, and (d) cooperative-game attribution (Shapley, interaction, deletion, directional) under one evidence-gated pipeline whose explicit rule is that embedding-based orderings must be reconfirmed on real held-out evaluation before use. We assert this narrowly for the specific combination as implemented in this repository – not for the research space generally, and not as any claim of intrinsic novelty for any single component.

> **Correction of record.** The project's own research notes (`research-landscape.md` under `docs/current/`) cite APE as arXiv:2210.03493. A live arXiv API check for this preprint shows `2210.03493` is instead *Automatic Chain of Thought Prompting in Large Language Models* [Zhang et al., 2022]; the correct APE identifier is arXiv:2211.01910 [Zhou et al., 2022]. The present paper cites the corrected ID; the note file should be updated.

The toolkit's vocabulary, quoted from its source where possible.

A prompt candidate is reduced to a *state* in a synthetic embedding space. Formally the scaffold defines:

- **Attractor** – a named point $\mathbf{a}_k \in \mathbb{R}^D$ representing a target behavioral disposition. The shipped set is `VERIFY_PRIOR`, `CREATIVE`, `CAUTIOUS`, `AUTHORITATIVE`, `TRUSTING`, with default dimension $D = 32$.
- **Prompt** – a dataclass of text, a token count, and an embedding bias; optional scalar `nonlinearity` modulates resonance.
- **Final state**. The candidate's latent landing point is

$$\mathbf{s} = \mathbf{b} + \mathbf{r}\left(1 + \nu \log(1 + \|\mathbf{r}\|)\right) \cdot \frac{1}{1 + 0.15\max(\text{tokens} - 2, 0)}, \tag{1}$$

where $\mathbf{b}$ is the baseline (here $\mathbf{0}$) and ν is the nonlinearity. Longer prompts are scaled down (an efficiency factor), which is the source of the density intuition.

- **Basin probability**. Cosine distance from $\mathbf{s}$ to each attractor center, then a max-subtracted softmax at temperature $T = 10$:

$$p_k(\mathbf{s}) = \frac{\exp(-d_{\cos}(\mathbf{s}, \mathbf{a}_k)/T)}{\sum_j \exp(-d_{\cos}(\mathbf{s}, \mathbf{a}_j)/T)}. \tag{2}$$

  Together with the attractor center $\mathbf{a}_0$ (the *target*), these yield the per-candidate metric vector, all from a single deterministic run: `target_activation` $= p_0(\mathbf{s})$; `reliability` $= \text{mean}_i\, p_0(\mathbf{s} + \varepsilon_i)$ for $\varepsilon_i \sim \mathcal{N}(0, 0.15)$; `distortion` $= 1 -$ `reliability` (a deliberately-synthetic proxy, openly the toy case the project's own notes criticize); `off_target` $= \max_{k \neq 0} p_k(\mathbf{s})$; `variance_penalty` $= \text{std}_i\, p_0(\mathbf{s}+\varepsilon_i)$; `distribution_shift` $= W_1(p_0(\text{noisy}), p_0(\text{noisy baseline}))$; `instruction_density` $=$ `target_activation`$\cdot$`reliability`$/\max(\text{tokens}, 1)$; and the scalar `score` $= p_0 \cdot$ `reliability` $-$ w_{off} `off_target` $-$ w_{var} `variance_penalty`. Finally `synAlign` is the cosine of $\mathbf{s}$ onto the target center: it is the *synthetic alignment signal*, an ordering heuristic for the machinery, and is labeled as such rather than as evidence.

The *selection layer* refuses to reify any single scalar. It computes an empirical distortion-rate curve (best distortion at or below each token budget), the Pareto-non-dominated subset over $\langle$`target_activation`, `reliability`$\rangle$ maximized and $\langle$`tokens`, `off_target`, `variance_penalty`$\rangle$ minimized, and a next-experiment recommendation by uncertainty-weighted gain. Two of its objects matter most for the paper's thesis:

- **Hard gates**. A mapping of metric keys to bounds. A candidate that violates any key is *excluded unconditionally*; score cannot rescue it. This is the operational form of "some constraints are $\leq \delta_{\text{critical}}$, not tradeable objectives."
- **Rate-distortion frontier**. An *empirical* lower envelope over measured tokens, not Shannon's theoretical bound (the distinction is drawn explicitly against Nagle et al. [2024] in Section).

The *attribution layer* provides component-level estimands over any supplied value function v: cooperative-game Shapley values (exact permutation enumeration when the coalitions are small, reproducible Monte-Carlo otherwise), the second-order inclusion-exclusion interaction term $v(\{i,j\}) - v(\{i\}) - v(\{j\}) + v(\emptyset)$, a paired deletion estimand, and a directional projection of an observed state change onto a declared target direction. Critically, these estimators are *value-function agnostic*: their conclusions are only as valid as the v supplied. Shapley over the synthetic cosine score is not Shapley over held-out architectural behavior.

The *context layer* is an order-aware variant of the same idea: a token sequence is position-encoded, passed through multi-head scaled-dot-product *causal* self-attention, pooled to a normalized contextual vector, and evaluated with the same attractor metrics; a KV-cache memoizes the per-token projections so repeated evaluations are bit-identical to a from-scratch pass but cheaper. The shipped decomposition (from the package layout and modules, not re-derived):

- `prompt_optimization.metrics` – attractor simulation and evaluation (`basin.py`), uncertainty (`uncertainty.py`: bootstrap interval, conformal radius, 1-Wasserstein, robust summary), and selection (`optimize.py`: Pareto, rate-distortion, metamorphic consistency, hard gates, next-experiment).

- `prompt_optimization.attribution` – `shapley.py`, `interaction.py`, `direction.py`.
- `prompt_optimization.context` – `attention.py`, `sequence.py`, `cache.py` (order-aware token path).
- `prompt_optimization.cli.py` – the `prompt-opt` surface.

Figure 1 shows the intended control flow. The single most important architectural fact – and the paper's central evaluation finding – is the vertical boundary inside the figure: everything above it is *built and tested on synthetic data*; everything below it is *described in prose only*. The scaffold can *propose*; nothing in the shipped code is a seam at which a real held-out model evaluation could *dispose*.

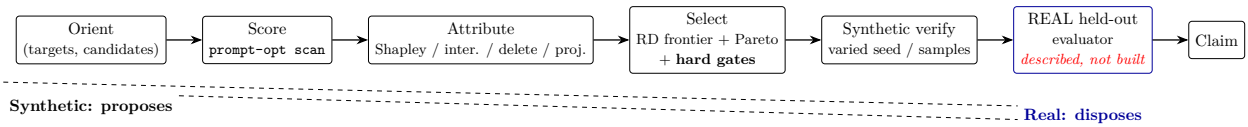


Figure 1: The intended pipeline. The dashed vertical boundary separates the synthetic scaffold (built and tested) from the real held-out evaluator (prose only in the current codebase).

The pipeline underneath the CLI is deliberately deterministic and reproducible: the same seed and sample count reproduce the same metric vector, and a KV-cache guarantees bit-identical results between cached and from-scratch context passes (verified by test). Nothing in the architecture pretends that the synthetic attractor is a stand-in for a production model; the disclaimers in the CLI output, the README, and the companion skill all say so in the same words: *synthetic scaffold: candidates only; real held-out evaluation disposes before any claim.* The public entry point is a stdlib-only command line (`prompt-opt scan` / `prompt-opt plan`), installed as a console script from `pyproject.toml`; `python -m prompt_optimization ...` is equivalent. The only third-party runtime dependency is NumPy.

Command	Flags	When to use
`prompt-opt scan <paths...>`	`--d-model` (must match attractor dim), `--target`, `--seed`, `--samples`, `--limit`, `--json`	per-candidate metric vector; the `synAlign` column is the synthetic alignment signal
`prompt-opt plan <paths...>`	scan flags **plus** `--gate KEY=BOUND` (repeatable), `--gate-op maxmin|`, `--json`	hard-gate veto, then RD frontier + Pareto + next experiment

Directories are walked recursively; binaries, hidden files, and derived dirs (`.git`, `.venv`, `__pycache__`, `node_modules`) are skipped. "Tokens" is a tokenizer-free estimate (word/punctuation count), explicitly not a real tokenizer.

Table 1: The `prompt-opt` surface.

Two implementation details matter for the evaluation section. First, the *hard-gate* wiring was found buggy during preparation of this preprint (the parser stored `-gate` under `args.gate`, but planning read `args.gates`, so gates were silently never applied); the fix is in the tree and covered by regression tests. Second, the human-readable scan table is still *sorted by* ***`synAlign`*** (cosine

similarity), most-aligned first – an explicit remainder of the exact ranking role the project's own thesis says should be demoted; the selection layer does not use it, but the display does.

The Python API re-exports the public surface at the top level (`from prompt_optimization import evaluate, pareto_frontier, shapley_values, ContextCache, ...`), so all downstream code paths (metrics, attribution, context) are importable without reaching into subpackages. This section is a *real transcript* – the tool was actually run on 2026-08-28 against an independent target repository, the `specimen` Claude Code skill (`Z:\Specimen`), with the raw output files preserved on disk under `output/specimen test 2/` in this project. It is not a hand-traced walkthrough; the commands below are the ones that produced the cited artifact files.

Context. `specimen` is an evidence-gated preprint skill: given a project, it produces a paper that never claims more than the project's evidence supports. Its repository contains seven candidate files (a `SKILL.md`, a README, four `references/*.md` discipline documents, and one helper script `scripts/arxiv_lookup.py`). The question put to the scaffold was the natural one: *of these candidate formulations, which does the pipeline favor under the default `VERIFY_PRIOR` attractor, and is any selection approximation to a behavioral preference?*

```
# 1. score every candidate file
prompt-opt scan --json Z:\Specimen
# 2. select over all candidates (permissive gate)
prompt-opt plan --gate distortion=0.8010 Z:\Specimen
# 3. select with a hard gate that only the cheapest point clears
prompt-opt plan --gate distortion=0.80065 Z:\Specimen
# 4. select with a strict hard gate every candidate fails
prompt-opt plan --gate distortion=0.7700 Z:\Specimen
```

Transcript highlights (abridged; full artifact files `scan.json`, `plan*.txt/json` are in `output/specimen test 2/`):

- The scan reported near-uniform metrics: the nontrivial spread was in `synAlign` (0.128 for README down to -0.048 for `latex-scaffold.md`); `target_activation` sat in $[0.199, 0.202]$, `reliability` in $[0.199, 0.199]$, and therefore `distortion` in $[0.8006, 0.8007]$ for every candidate. The frontier – best distortion at each token budget – collapsed to a single repeated point: `README.md` at 303 tokens, distortion 0.8006, at *every* budget.
- The Pareto-non-dominated set was `README.md` (303 tok, act 0.202), the `section-template.md` in `references/` (774 tok), and `scripts/arxiv_lookup.py` (903 tok). The recommended next experiment was `README.md` (highest uncertainty-weighted gain, here tiny).
- With a hard gate at `distortion=0.80065`, exactly one candidate survives — `README.md`; the other six are reported as `blocked`.
- With a hard gate at `distortion=0.77`, *no* candidate survives, and the tool prints exactly the honest negative result: `"no candidate passes the hard gates ...; no supported improvement is claimed."`

How to read this. The transcript demonstrates that the machinery works as specified: it scans, it gates (hard), it builds a frontier, and — the most specimen-native behavior — it is willing to *lose* loudly rather than force a winner. It equally demonstrates that, at this attractor layout and dimension, the scaffold has almost no discrimination: a narrow rounding margin decides which single candidate survives a meaningful gate, and the frontier is a constant. That is a property of

the *synthetic scaffold*, not of the `specimen` skill; under no reading are these numbers evidence that `README.md` "causes more verify-prior behavior." They are evidence that the ordering/selection logic runs and respects its gates.

This section is written to the discipline of *evidence-status*: it reports what evidence exists, classified by what each piece *licenses*, and it ends with an explicit claim boundary. The reminder that opens it is the discipline's reflexive principle applied to this paper: a preprint arguing that engineering claims should be evidence-gated must itself be evidence-gated, or it is doing exactly the thing it warns about. The sections above are machinery; the claims below are what the reader may actually rely on.

0.1 Step 0: first-party evidence produced for this preprint

1. **Test suite, re-run** on 2026-08-28: `46 passed` (5 test modules). Exercised through the package's public interface, including determinism, normalization, metric bounds, zero-token handling, Pareto / rate-distortion invariants, hard-gate wiring, JSON emission purity, attention masking, and KV-cache bit-identity.

2. **CLI usage transcripts** on 2026-08-28, preserved on disk (`output/specimen test 2/`): `scan` and `plan` runs with three gate regimes against the `specimen` skill repository (Section 6).

0.2 Step 1–2: evidence inventory and classification

Evidence	Status	What it licenses
Pre-registered protocol	none	—
Dated results log / case study	none	—
Executed behavior (test suite)	46/46 pass, 2026-08-28	the exercised invariants behaved this way; NOT general correctness
Executed behavior (CLI runs)	gate regimes behaved as specified, dated	ordering/gating logic runs and respects hard gates; NOT "these prompts are better"
Static structure (lint/type checks)	none declared	—
Empirical measurement (real model)	none	—
Attribution outputs	subroutines covered by 3 tests	the estimators compute their defined quantities; NOT causal findings about prompts
Model/LLM judgment	none collected	—
Human decision	one external adversarial skill review (2026) + this preprint	an accountable person accepted the design stance; NOT that the stance is validated

Table 2: Evidence classification for `prompt-optimization` (2026-08-28).

A single row must not be written up as if it answered a question from a different row. The test suite licenses *"the exercised cases behaved this way"* and nothing about whether the toolkit improves any real prompt. The CLI transcripts license *"the selection logic ran and respected its gates"* and nothing about the `specimen` repository being optimized.

0.3 Failures of record (reported, not papered over)

- **Stale/mislabeled citation in the project's own notes.** `research-landscape.md` (under `docs/current/`) cites APE as arXiv:2210.03493; the live arXiv API identifies that ID as *Automatic Chain of Thought Prompting*, and APE is 2211.01910 (Section 2).
- **Hard-gate wiring bug (fixed).** Planning read `args.gates` while parsing stored `args.gate`, so `plan -gate ...` silently did nothing. Found while preparing this preprint's worked example; fixed; two regression tests added.
- **JSON pollution (fixed).** The strict-gate JSON path printed a human line to stdout before the JSON document, corrupting machine output. Fixed and covered by the same regression tests.
- **Missing top-level export (fixed).** `apply_hard_gates` was importable from the `metrics` subpackage but not re-exported at the package root, breaking a test import.

These are reported *because* a fix list is the deliverable of an honest evaluation status section: the first item is a documentation defect still in the tree; the other three are defects already corrected, kept as findings of record.

0.4 Claim boundary

The mechanisms above are individually grounded in a passing test suite and in the project's own design notes. They do not establish that `prompt-optimization` *improves any outcome on any real language model, nor that any candidate ranking it produces reflects behavioral equivalence under held-out evaluation. That is an open, and not yet even designed, empirical question — and, as the pipeline architecture itself insists, claimed performance requires a real held-out evaluator to dispose of synthetic proposals before the claim exists.*

The absence this section documents is not a defect in the writeup; per the skill's own philosophy it is the finding: the toolkit's value today is as a deterministic harness for developing ordering and selection logic, and its next milestone is a real evaluation seam, not more math.

The synthetic evaluator is a stagehand, not a measure. `basin_probabilities` is a softmax over cosine distances from content-derived embeddings to five hand-placed centers at $D = 32$; `distortion = 1 - reliability` is, by the project's own admission in `docs/current/research-landscape.md`, a toy proxy until a task-specific behavioral distortion is defined. No threshold in this paper (including the `0.80065` gate in the worked example) is calibrated against anything.

Discrimination collapse. In the worked example every candidate's `distortion` landed in $[0.8006, 0.8007]$: the metric space is effectively flat at the tested dimension/layout, the rate-distortion frontier is a constant, and gate membership is decided by rounding. A scaffold that cannot separate candidates cannot be trusted to rank candidates; this is a concrete threat to the validity of *any* ordering the current scaffold emits and should be on the front page of any future real test.

Embedding similarity still holds an ordering role. The human-readable scan table is sorted by `synAlign` (cosine), the exact "most-aligned first" posture the project's thesis argues against (Section 5). The selection layer ignores it, but a naive reader of the scaffold output can still be misled into reading the display's first row as the "answer."

No real-evaluator seam. There is no interface, protocol, or CLI flag through which a real held-out model evaluation can be supplied; the "real held-out evaluation disposes" language exists only in README/CLI prose (Section 4, Figure 1). A skill (the companion `specimen-prompt-eval`) that tells an agent to run step "verify (real)" currently has no concrete command to give.

Attribution is value-function-agnostic but under-plumbed. The estimators are correct for their definitions, but nothing ships an example value function, and no CLI path exists; an agent cannot currently exercise attribution without reading package source.

Other scope boundaries. Tokenizer-free token estimates; a Windows-tested, single-numpy-dependency environment; no lint/type-check step committed; no pre-registration (risk of post-hoc fitting to attractor placement); the worked example was run before the gate bug was fixed, so its permissive-gate numbers are the fixed behavior re-verified on 2026-08-28. The evaluation gap in Section 7 maps to concrete work, ordered by the leverage it would buy. Each item is a *proposal*, not a built feature.

1. Build the real-evaluator seam (highest leverage). Add a minimal `Evaluator` protocol and a `prompt-opt plan -backend <adapter>` hook so a real held-out model (or a human-judged rubric) can be plugged into the pipeline's *dispose* position. Then make the pipeline refuse to emit a "selection" as a claim unless a real run is present. This is the review's centerpiece: *synthetic optimizer proposes; real held-out evaluator disposes.* Without the seam, the pipeline's own central rule is aspirational.

2. Expose attribution through the CLI. A `prompt-opt attribute` subcommand with a shipped example value function (and its limitations) would make the attribution layer executable by the same agents who run `scan`/`plan`, closing the gap that currently forces reading source.

3. De-rank the display. Stop sorting the scan table by `synAlign`; sort by a behavioral or score-based ordering, or leave rows in corpus order. This closes the gap between the thesis and the last place embedding similarity still outranks behavior.

4. Pre-register before measuring. Draft and date a protocol (task population, comparator, outcome measures, stopping rule, a commitment to report null and adverse results) *before* collecting real-model data – per the evidence-status discipline, to convert "we have no evidence yet" into a checkable plan rather than a permanent hedge.

5. Run two model families on held-out tasks. Evaluate the surviving candidates on unseen tasks across at least two model families, report bootstrap intervals around each Pareto front point, and keep optimizer and evaluator disjoint (the OPRO scale-limits warning, Zhang et al. [2024]). Only then fit any activation probe.

6. Housekeeping. Fix the APE identifier in `docs/current/research-landscape.md`; add a committed lint/type-check step; consider a task-specific behavioral distortion measure to replace `1 - reliability`.

None of items 1–3 requires new mathematics; all three are architectural, which is exactly where this project's next step lies. `prompt-optimization` is a deterministic, reproducible scaffold for developing the ordering and selection logic of evidence-gated prompt optimization: name behavioral attractors, score candidates, attribute differences, select on a distortion-rate and Pareto frontier with mandatory hard gates that no score overrides, and *never* promote a synthetic ordering to a performance claim. Its evaluation status is honest rather than flattering: the shipped evidence is a passing invariant suite and real CLI transcripts on a synthetic scaffold; there is no real held-out model measurement, no pre-registered protocol, and no seam yet at which a real evaluator could dispose of synthetic proposals. Those are findings, and the concrete fix list that closes them is the next contribution this line of work should make.

References

Victor Akinwande, Yiding Jiang, Dylan Sam, and J. Zico Kolter. Understanding prompt engineering may not require rethinking generalization. 2023.

Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. 2021.

Lukasz Bartoszcze, Sarthak Munshi, Bryan Sukidi, Jennifer Yen, Zejia Yang, David Williams-King, Linh Le, Kosi Asuzu, and Carsten Maple. Representation engineering for large-language models: Survey and research challenges. 2025.

Mateusz Bystroński, Grzegorz Piotrowski, Nitesh V. Chawla, and Tomasz Kajdanowicz. Latentprompt: Optimizing prompts in latent space. 2025.

Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution. 2023.

Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. Evoprompt: Connecting llms with evolutionary algorithms yields powerful prompt optimizers. 2023.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Llmlingua: Compressing prompts for accelerated inference of large language models. 2023.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. Dspy: Compiling declarative language model calls into self-improving pipelines. 2023.

Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. 2019.

Kenneth Li, Oam Patel, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. Inference-time intervention: Eliciting truthful answers from a language model. 2023a.

Zekun Li, Baolin Peng, Pengcheng He, and Xifeng Yan. Evaluating the instruction-following robustness of large language models to prompt injection. 2023b.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in gpt. 2022.

Alliot Nagle, Adway Girish, Marco Bondaschi, Michael Gastpar, Ashok Vardhan Makkuva, and Hyeji Kim. Fundamental limits of prompt compression: A rate-distortion framework for black-box language models. 2024.

Nina Panickssery, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Matt Turner. Steering llama 2 via contrastive activation addition. 2023.

Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. Automatic prompt optimization with "gradient descent" and beam search. 2023.

Antonio Sabbatella, Andrea Ponti, Antonio Candelieri, Ilaria Giordani, and Francesco Archetti. A bayesian approach for prompt optimization in pre-trained language models. 2023.

Taylor Shin, Yasaman Razeghi, Robert L. Logan, Eric Wallace, and Sameer Singh. Autoprompt: Eliciting knowledge from language models with automatically generated prompts. 2020.

Naftali Tishby, Fernando C. Pereira, and William Bialek. The information bottleneck method, 2000.

Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J. Vazquez, Ulisse Mini, and Monte MacDiarmid. Steering language models with activation engineering. 2023.

Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. The instruction hierarchy: Training llms to prioritize privileged instructions. 2024.

Zhaofeng Wu, Linlu Qiu, Alexis Ross, Ekin Akyürek, Boyuan Chen, Bailin Wang, Najoung Kim, Jacob Andreas, and Yoon Kim. Reasoning or reciting? exploring the capabilities and limitations of language models through counterfactual tasks. 2023.

Sang Michael Xie, Aditi Raghunathan, Percy Liang, and Tengyu Ma. An explanation of in-context learning as implicit bayesian inference. 2021.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. 2023.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. Textgrad: Automatic "differentiation" via text. 2024.

Tuo Zhang, Jinyue Yuan, and Salman Avestimehr. Revisiting opro: The limitations of small-scale llms as optimizers. 2024.

Zhuosheng Zhang, Aston Zhang, Mu Li, and Alex Smola. Automatic chain of thought prompting in large language models. 2022.

Junjie Zhou and Zhijian Ou. Bayespo: Bayesian prompt optimization via parallel-tempered gradient-guided discrete mcmc. 2026.

Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. Large language models are human-level prompt engineers. 2022.

Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, Shashwat Goel, Nathaniel Li, Michael J. Byun, Zifan Wang, Alex Mallen, Steven Basart, Sanmi Koyejo, Dawn Song, Matt Fredrikson, J. Zico Kolter, and Dan Hendrycks. Representation engineering: A top-down approach to ai transparency. 2023.